

Seminar 1: Graphs and Isomorphism

Carlos Buitrago

Table of contents

1	Graphs	2
1.1	Degree of a Vertex	3
1.2	Theorem (Handshake Lemma)	3
2	Graph Isomorphism	4
2.1	Invariants of Isomorphism	5
3	Connected Graphs	5
3.1	Trees	5
3.1.1	Theorem (Equivalent Characterizations of Trees)	6
3.2	Complement of a Graph	7
3.3	Self-Complementary Graphs	8
3.3.1	Proposition	8
3.3.2	Connectivity of Complements	9
4	Ramsey Theory	9
4.1	Definition	9
5	How to Verify Whether Two Graphs Are Isomorphic	10
5.1	Step 1. Compare the Degree Sequences	10
5.2	Step 2. Compare Local Structures	10
5.3	Step 3. Match Distinguished Vertices	11
5.4	Step 6. Construct an Explicit Bijection	11
6	Beyond Graph Invariants: The Graph Isomorphism Problem	12
6.1	The Graph Isomorphism Problem	12
6.1.1	Input	12
6.1.2	Question	12
6.2	A Naive Algorithm	13
6.2.1	How Long Would Brute Force Take?	13

6.3	Complexity-Theoretic Status	14
6.4	The Weisfeiler–Leman Method	14
6.5	Babai’s Breakthrough (2015)	15
6.6	Special Classes of Graphs	15
6.7	Modern Algorithms	16
6.8	State of the Art	16
7	Why Should Data Scientists and Computer Scientists Care About Graph Theory?	17
7.0.1	Social Networks	17
7.0.2	Recommendation Systems	18
7.0.3	The Internet	18
7.0.4	Maps and Navigation	19
7.0.5	Machine Learning on Graphs	19
7.0.6	Graph Neural Networks (GNNs)	20
7.0.7	Molecules and Drug Discovery	20
7.0.8	Knowledge Graphs and Large Language Models	21
7.0.9	Computer Networks and Cybersecurity	21
7.0.10	Biology and Medicine	21

In this seminar we introduce the basic notions of graph theory required for studying graph isomorphism, connectedness, graph complements, and elementary Ramsey theory.

1 Graphs

A **simple graph** is a pair

$$G = (V, E),$$

where

- V is a finite set of vertices;
- $E \subseteq \binom{V}{2}$ is a set of unordered pairs of distinct vertices called **edges**.

Throughout the seminar we consider only finite simple graphs.

If $|V| = p$, $|E| = q$, then G is called a (p, q) -**graph**.

The complete graph on n vertices is denoted by K_n .

1.1 Degree of a Vertex

The **degree** of a vertex v is the number of edges incident with v . It is denoted by $d(v)$.

The list

$$(d(v_1), \dots, d(v_n))$$

is called the **degree sequence** of the graph. Usually the degree sequence is written in non-increasing order.

1.2 Theorem (Handshake Lemma)

For every graph G ,

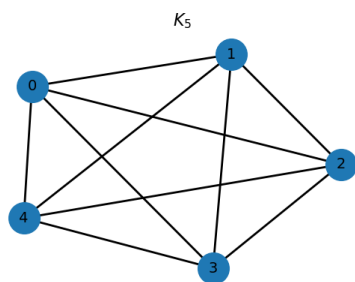
$$\sum_{v \in V(G)} d(v) = 2|E(G)|.$$

A sequence $(d_1, \dots, d_n)$ is called **graphical** if there exists a graph whose degree sequence is equal to this sequence.

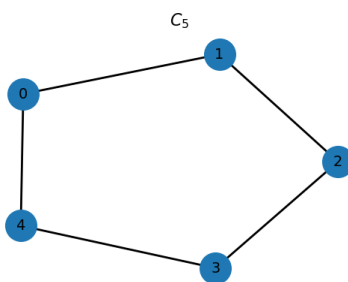
The Handshake Lemma immediately implies a necessary condition:

$$d_1 + \dots + d_n$$

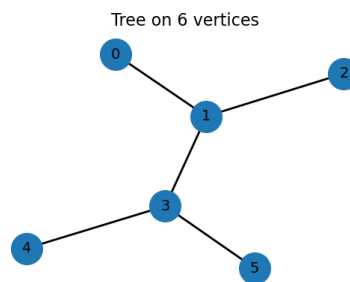
must be an even number. This condition is necessary but not sufficient. For example, $(4, 4, 4, 4, 2)$ is not graphical.



Degree sequence: $[4, 4, 4, 4, 4]$
 $|E| = 10$, sum degrees = 20
 sum degrees = $2|E| = 20$



Degree sequence: $[2, 2, 2, 2, 2]$
 $|E| = 5$, sum degrees = 10
 sum degrees = $2|E| = 10$



Degree sequence: $[3, 3, 1, 1, 1, 1]$
 $|E| = 5$, sum degrees = 10
 sum degrees = $2|E| = 10$

2 Graph Isomorphism

Graphs $G = (V, E)$ and $H = (W, F)$ are called **isomorphic** if there exists a bijection $\varphi : V \rightarrow W$ such that

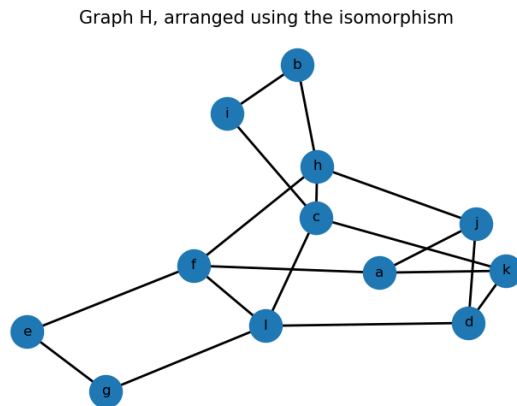
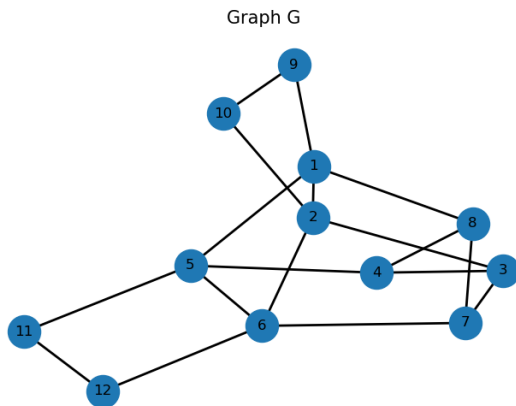
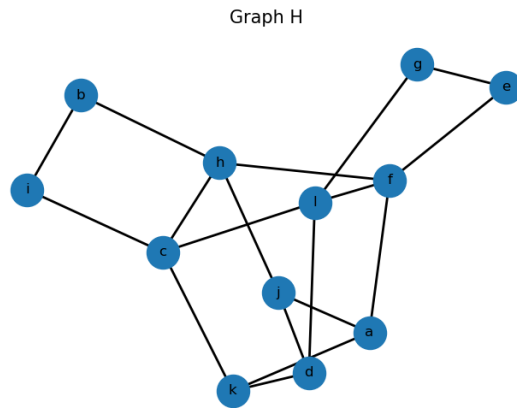
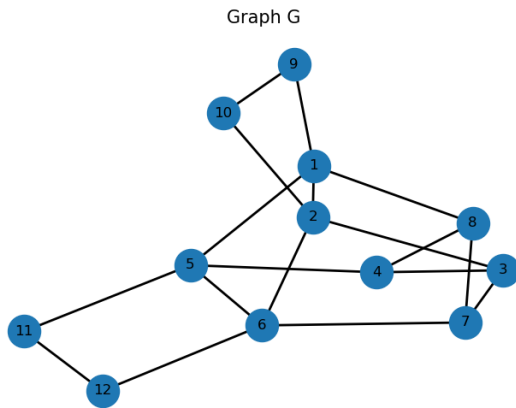
$$uv \in E \iff \varphi(u)\varphi(v) \in F.$$

The map φ is called an **isomorphism**.

We write $G \cong H$.

Intuitively, two graphs are isomorphic if one can be obtained from the other merely by relabelling the vertices.

Two graphs on 12 vertices that look different, but are isomorphic



Isomorphism: $\varphi(1) = h, \varphi(2) = c, \varphi(3) = k, \varphi(4) = a, \varphi(5) = f, \varphi(6) = l, \varphi(7) = d, \varphi(8) = j, \varphi(9) = b, \varphi(10) = i, \varphi(11) = e, \varphi(12) = g$

2.1 Invariants of Isomorphism

A quantity that is preserved under isomorphism is called a **graph invariant**.

If

$$G \cong H,$$

then the following quantities coincide:

- number of vertices;
- number of edges;
- degree sequence;
- number of connected components;
- number of cycles of each length.

Graph invariants are useful for proving that two graphs are not isomorphic.

3 Connected Graphs

A **path** in a graph is a sequence of vertices

$$v_0, v_1, \dots, v_k$$

such that every consecutive pair of vertices is connected by an edge.

A graph is called **connected** if every pair of vertices can be joined by a path.

A maximal connected subgraph is called a **connected component**.

Every graph can be uniquely decomposed into connected components.

3.1 Trees

A graph is called a **tree** if it is connected and contains no cycles.

3.1.1 Theorem (Equivalent Characterizations of Trees)

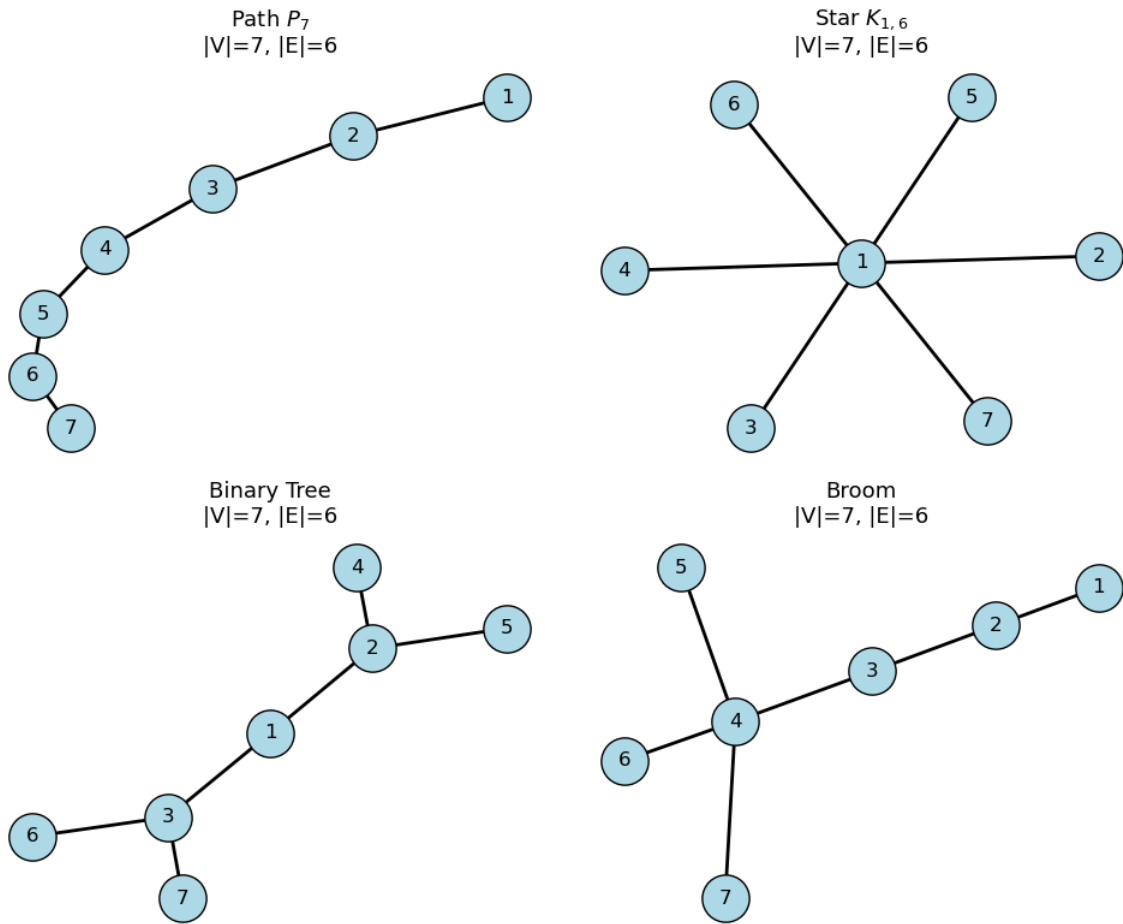
Let $G = (V, E)$ be a graph with n vertices. The following statements are equivalent:

1. G is a tree.
 2. G is connected and has exactly $(n-1)$ edges.
 3. G is acyclic and has exactly $(n-1)$ edges.
 4. G is connected, and every edge is a bridge (removing any edge disconnects the graph).
 5. G is acyclic, and adding any missing edge creates exactly one cycle.
 6. For every pair of distinct vertices (u, v) , there exists a unique path connecting (u) and (v) .
 7. G is connected and removing any edge destroys connectivity.
 8. G is acyclic and adding any edge destroys acyclicity.
-

These characterizations are often used interchangeably. Depending on the problem, it may be easier to verify one of them rather than the original definition.

For example:

- To prove a graph is a tree, it is often easiest to show that it is connected and has $(n-1)$ edges.
- To show that a graph is **not** a tree, it is enough to find either a cycle or two vertices that are not connected.
- The unique-path characterization is particularly useful in algorithms and network design.



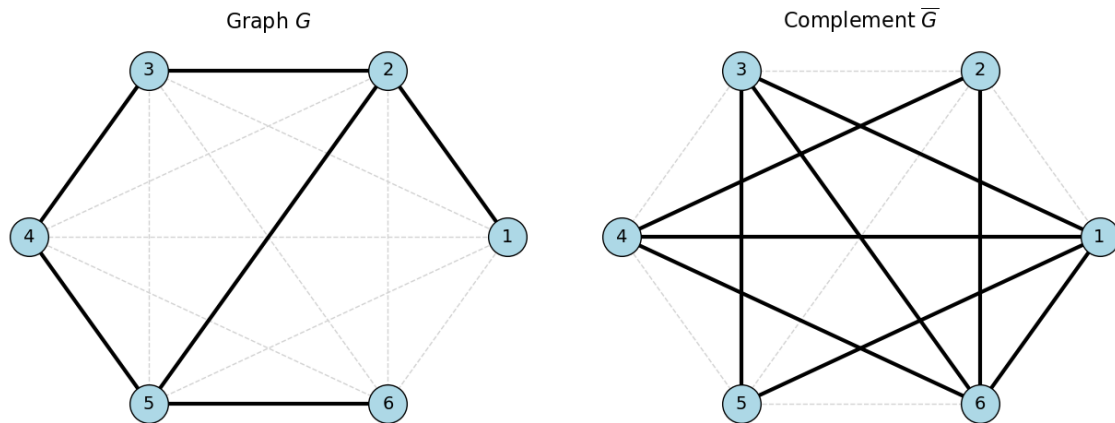
3.2 Complement of a Graph

Let $G = (V, E)$ be a graph. The **complement** of G is the graph $\overline{G} = (V, \overline{E})$, where

$$uv \in \overline{E} \iff uv \notin E.$$

Thus

$$E(\overline{G}) = E(K_n) \setminus E(G).$$



3.3 Self-Complementary Graphs

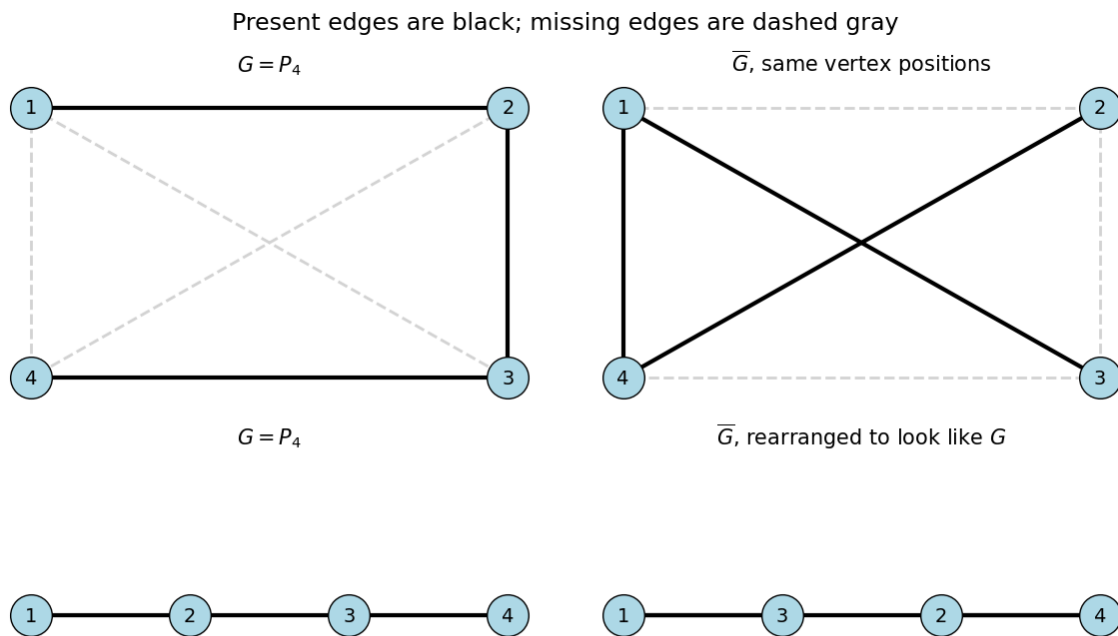
A graph is called **self-complementary** if

$$G \cong \overline{G}.$$

3.3.1 Proposition

If G is self-complementary and has n vertices, then

$$|E(G)| = \frac{1}{2} \binom{n}{2}.$$



3.3.2 Connectivity of Complements

3.3.2.1 Theorem

For every graph G , at least one of the graphs G and $\overline{G}$ is connected.

Corollary: Every self-complementary graph is connected.

4 Ramsey Theory

Ramsey theory studies the appearance of order in sufficiently large structures.

4.1 Definition

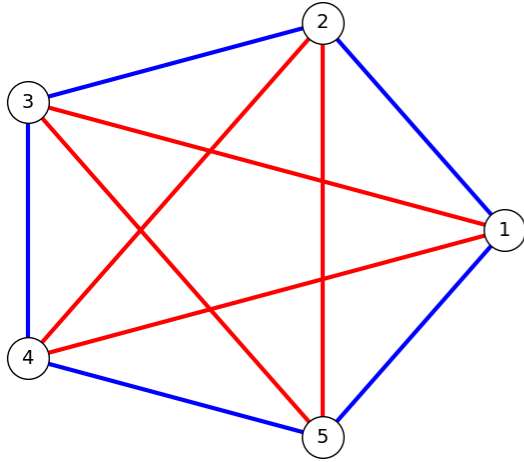
The **Ramsey number** $R(s, t)$ is the smallest integer n such that every red-blue colouring of the edges of K_n contains either

- a red copy of K_s , or

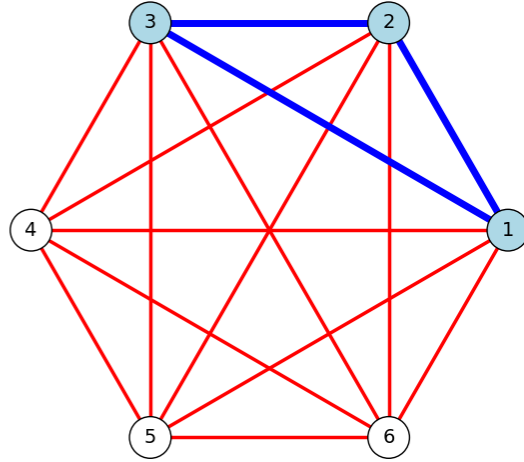
- a blue copy of K_t .

Ramsey's theorem: $R(3, 3) = 6$

K_5 : no monochromatic triangle



K_6 : monochromatic blue triangle



5 How to Verify Whether Two Graphs Are Isomorphic

Determining whether two graphs are isomorphic is one of the fundamental problems of graph theory.

The trivial step is to compare the number of vertices and edges.

5.1 Step 1. Compare the Degree Sequences

Compute the degrees of all vertices and sort them in non-increasing order.

If the resulting sequences differ, the graphs are not isomorphic.

5.2 Step 2. Compare Local Structures

Even if two graphs have the same degree sequence, they may still be non-isomorphic.

Useful quantities include:

- number of connected components;

- number of triangles;
- number of cycles of a given length;
- number of vertices of each degree;
- adjacency relations between high-degree vertices.

If any of these quantities differ, the graphs are not isomorphic.

5.3 Step 3. Match Distinguished Vertices

Suppose the graphs have passed all previous tests.

Vertices with unique properties must correspond to each other under any isomorphism.

For example, if a graph contains exactly one vertex of degree 4, then this vertex must be mapped to the unique degree-4 vertex of the other graph.

This often greatly reduces the number of possible bijections.

5.4 Step 6. Construct an Explicit Bijection

After identifying corresponding vertices, attempt to define a bijection

$$\varphi : V(G) \rightarrow V(H).$$

One must verify that

$$uv \in E(G) \iff \varphi(u)\varphi(v) \in E(H)$$

for every pair of vertices.

If such a bijection exists, then $G \cong H$.

6 Beyond Graph Invariants: The Graph Isomorphism Problem

The method based on graph invariants is extremely useful in practice. However, there exist many pairs of non-isomorphic graphs that share all common invariants such as:

- number of vertices;
- number of edges;
- degree sequence;
- number of connected components;
- number of cycles of various lengths.

Consequently, a more systematic approach is required.

This leads to one of the most famous algorithmic problems in graph theory.

6.1 The Graph Isomorphism Problem

6.1.1 Input

Two graphs

$$G = (V, E), \quad H = (W, F)$$

with $|V| = |W| = n$.

6.1.2 Question

Does there exist a bijection $\varphi : V \rightarrow W$ such that

$$uv \in E(G) \iff \varphi(u)\varphi(v) \in E(H)?$$

If such a bijection exists, the graphs are isomorphic.

6.2 A Naive Algorithm

A brute-force approach consists of checking all possible bijections

$$\varphi : V \rightarrow W.$$

Since there are $n!$ such bijections, the running time is approximately $O(n!)$.

Using Stirling's approximation,

$$n! \approx \sqrt{2\pi n} \left(\frac{n}{e}\right)^n,$$

which grows extremely rapidly.

6.2.1 How Long Would Brute Force Take?

Suppose, very optimistically, that a modern computer can check 10^9 (one billion) bijections per second. Then the running time is approximately $\frac{n!}{10^9}$ seconds.

For example, $20! \approx 2.4 \cdot 10^{18}$.

Hence brute force would take approximately $\frac{20!}{10^9} \approx 2.4 \cdot 10^9$ seconds.

Since one year has about $3.15 \cdot 10^7$ seconds, this is approximately $\frac{2.4 \cdot 10^9}{3.15 \cdot 10^7} \approx 77$ years.

Thus, even for graphs with only 20 vertices, the naive algorithm is already infeasible.

The situation becomes much worse as n grows.

Number of vertices n	Number of bijections $n!$	Time at 10^9 checks/sec
10	$3.6 \cdot 10^6$	less than 1 second
15	$1.3 \cdot 10^{12}$	about 22 minutes
20	$2.4 \cdot 10^{18}$	about 77 years
25	$1.6 \cdot 10^{25}$	about $5 \cdot 10^8$ years
30	$2.7 \cdot 10^{32}$	about $8.4 \cdot 10^{15}$ years

Therefore, the brute-force method is not a serious algorithm for graph isomorphism, except for very small graphs.

This explains why more sophisticated methods are needed: instead of checking all possible vertex permutations, modern algorithms try to exploit graph structure, vertex refinement, symmetry, and group-theoretic ideas.

6.3 Complexity-Theoretic Status

The Graph Isomorphism problem occupies a unique position in complexity theory.

It is known that

$$\text{GI} \in \text{NP}.$$

Indeed, if an isomorphism exists, it can be verified in polynomial time.

Surprisingly, despite decades of research:

- no polynomial-time algorithm is known for general graphs;
- no proof exists that the problem is NP-complete.

Therefore Graph Isomorphism is one of the few classical problems whose precise complexity remains unknown.

6.4 The Weisfeiler–Leman Method

The most important practical idea is the **Weisfeiler–Leman refinement algorithm**.

The basic version proceeds as follows.

1. Assign every vertex the same color.
2. Repeatedly recolor each vertex according to the multiset of colors of its neighbors.
3. Continue until the coloring stabilizes.

If two graphs produce different color distributions, they cannot be isomorphic.

The algorithm runs in polynomial time.

In practice, it distinguishes most graphs encountered in applications.

6.5 Babai's Breakthrough (2015)

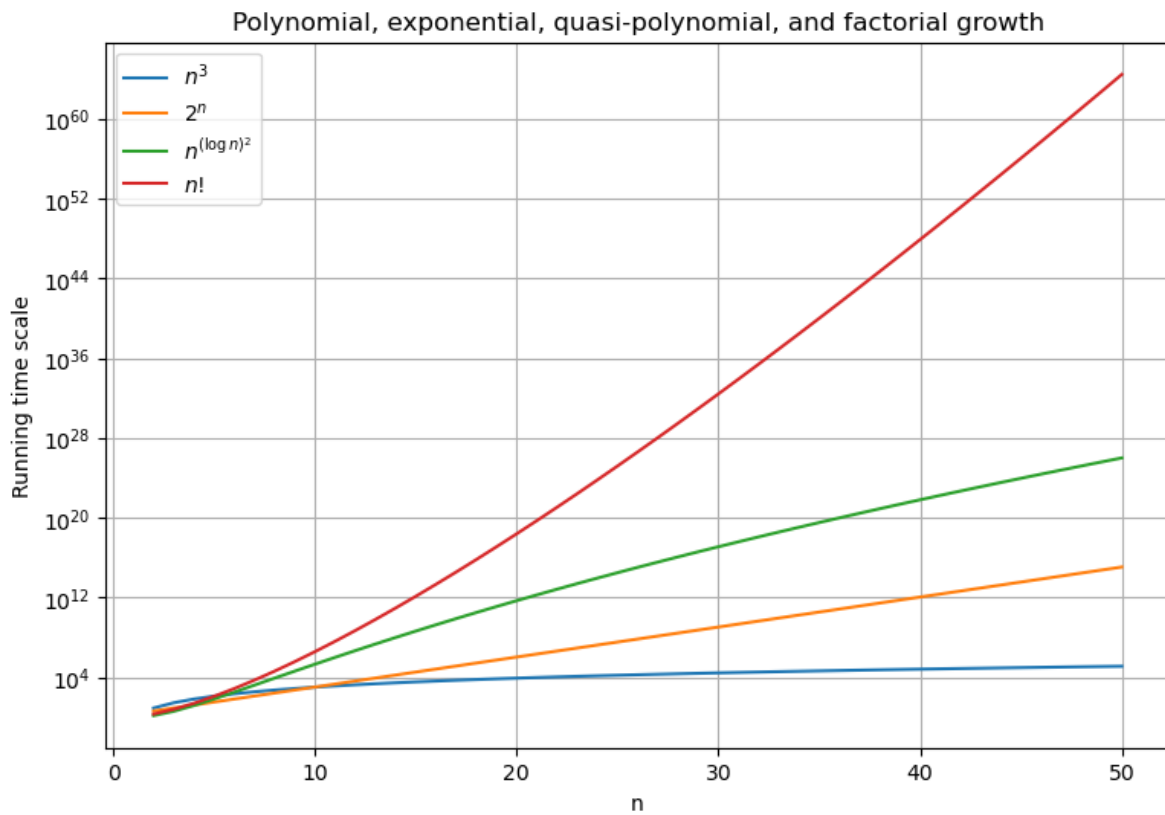
Babai proved that Graph Isomorphism can be solved in

$$2^{(\log n)^{O(1)}}$$

time.

Such running times are called **quasi-polynomial**.

They are significantly smaller than exponential running times such as 2^n but still larger than every polynomial n^k .



6.6 Special Classes of Graphs

Although the complexity of the Graph Isomorphism problem is unknown for arbitrary graphs, polynomial-time algorithms are known for many important graph families.

Examples include:

- trees;
- planar graphs;
- interval graphs;
- bounded-degree graphs;
- graphs of bounded treewidth.

For trees, graph isomorphism can be solved in

$$O(n)$$

time.

Consequently, the difficulty of the Graph Isomorphism problem comes not from sparse or highly structured graphs, but rather from graphs possessing many symmetries.

In practice, special graph classes arise frequently in computer science. Therefore, many real-world instances can be solved much faster than the worst-case theoretical bounds suggest.

6.7 Modern Algorithms

Modern graph isomorphism algorithms do not examine all possible vertex permutations.

Instead, they combine several ideas:

1. Vertex refinement techniques.
2. Partition refinement.
3. Canonical labeling.
4. Backtracking and pruning.
5. Group-theoretic methods.

The most influential practical approach is the Weisfeiler–Leman refinement procedure, which repeatedly partitions vertices into classes according to their local neighborhoods.

These methods eliminate the overwhelming majority of impossible vertex correspondences before any exhaustive search is attempted.

6.8 State of the Art

The current understanding of the Graph Isomorphism problem may be summarized as follows.

1. Graph Isomorphism belongs to the complexity class

NP.

2. No polynomial-time algorithm is known for arbitrary graphs.
3. No proof exists that Graph Isomorphism is NP-complete.
4. Babai's quasi-polynomial algorithm is the best known theoretical result for general graphs.
5. Polynomial-time algorithms are known for many important graph classes.
6. Practical software can routinely solve instances involving thousands or even millions of vertices.

Consequently, Graph Isomorphism occupies a unique position among classical computational problems: it is neither known to be easy nor known to be hard.

7 Why Should Data Scientists and Computer Scientists Care About Graph Theory?

At first sight, graph theory may look like a collection of abstract puzzles about vertices and edges. In reality, graphs are one of the most important mathematical models in modern computer science and data science.

The key idea is simple:

Whenever we have **objects** and **relationships between them**, we can model the situation as a graph.

7.0.1 Social Networks

In a social network:

- vertices = users;
- edges = friendships or interactions.

Questions such as

- Who are the most influential users?
- Which communities exist?
- How does information spread?

are naturally graph-theoretic questions.

Examples:

- Facebook friendship graph;
 - LinkedIn professional network;
 - Instagram follower network.
-

7.0.2 Recommendation Systems

Many recommendation systems can be viewed as graphs.

For example:

- users are vertices;
- movies are vertices;
- an edge connects a user to a movie they watched.

This creates a **bipartite graph**.

Companies such as Netflix, Amazon, Spotify, and YouTube use graph-based methods to recommend content.

7.0.3 The Internet

The Internet itself is a graph:

- vertices = routers or computers;
- edges = communication links.

Finding efficient routes for packets is a graph problem.

Algorithms such as:

- Breadth-First Search (BFS),
- Dijkstra's algorithm,
- shortest-path algorithms,

are used every day in networking.

7.0.4 Maps and Navigation

Road networks are graphs.

- vertices = intersections;
- edges = roads.

Applications:

- Google Maps;
- Yandex Maps;
- Waze;
- GPS navigation systems.

The problem

“How do I get from A to B as quickly as possible?”

is a shortest-path problem.

7.0.5 Machine Learning on Graphs

Many modern datasets are naturally graphs.

Examples:

- social networks;
- citation networks;
- biological networks;
- molecular structures.

Traditional machine learning methods often ignore relationships between observations.

Graph-based methods explicitly use these relationships.

One of the most active research areas today is:

7.0.6 Graph Neural Networks (GNNs)

A Graph Neural Network learns from both

- the features of a vertex;
- the structure of its neighborhood.

Applications include:

- recommendation systems;
 - fraud detection;
 - molecular property prediction;
 - social network analysis;
 - knowledge graphs.
-

7.0.7 Molecules and Drug Discovery

Chemists often represent molecules as graphs.

- vertices = atoms;
- edges = chemical bonds.

For example:

- predicting toxicity;
- predicting biological activity;
- discovering new drugs

can all be formulated as graph-learning problems.

Many recent AI systems for drug discovery rely heavily on graph representations.

7.0.8 Knowledge Graphs and Large Language Models

Search engines and modern AI systems often store information as graphs.

Examples:

- Google Knowledge Graph;
- Wikidata;
- semantic web databases.

A graph may contain facts such as

`(Marie Curie) -- discovered --> (Radium)`

or

`(Paris) -- capital of --> (France)`

Many modern AI systems combine language models with graph-based knowledge structures.

7.0.9 Computer Networks and Cybersecurity

Graph methods are widely used to detect:

- suspicious communication patterns;
- bot networks;
- fraud rings;
- money laundering schemes.

A fraud ring often appears as a characteristic graph structure.

7.0.10 Biology and Medicine

Examples of biological networks:

- protein interaction networks;
- gene regulatory networks;
- neural networks in the brain.

Researchers study these systems using graph-theoretic tools.